

Improving LLM-assisted automated test generation for Machine Learning in Python

Théo GINGUENÉ

July 27, 2026

Abstract

Your abstract.

Add abstract

Contents

1	Introduction	2
1.1	Problem	2
1.2	Contributions	3
2	Background and related work	3
3	Approach	3
3.1	New providers	4
3.2	Prompt engineering	4
3.3	Regex	5
3.4	Adding imports	6
3.4.1	pybind11 functions	7
3.5	Statement fixes	7
3.5.1	Asserts	7
3.5.2	Unary not	7
3.5.3	Equality between variable references	7
3.5.4	<code>x is y</code> (?)	7
3.6	Evaluation	7
3.7	Compressing context	7
4	Evaluation	8
4.1	Dataset	8
4.2	Experiment setup	8
4.2.1	Containerization	8
4.2.2	Configuration	9
4.3	Results	9
5	Discussion	9
5.1	Findings	9
5.2		9
5.3	Threats to validity	9
6	Conclusions	9
6.1	Summary	9
6.2	Future work	9
	Acknowledgments	9
	References	10

1 Introduction

Testing code is crucial to ensure its quality. Historically, this has been done manually, which can be tedious and time-consuming. To address this issue, automated test generation tools have been created. Many of them were first developed for Java, because of its widespread use in research and business, and its strong typing. One such example is Evosuite [1], which uses an evolutionary approach based on genetic algorithms. However, some specific fields of computer science like machine learning often use Python instead of Java. Using the same principles, Pynguin [2] was then developed to cover this gap in test generation tools for Python. The default algorithm these two tools use is called Dynamic Many-Objective Sorting Algorithm (DynaMOSA) [3]. DynaMOSA has been shown to outperform its predecessors [3], but a new successor recently emerged. With the emergence of Large Language Models (LLMs) also appeared LLM-assisted test generation. Combining LLMs and traditional approaches such as DynaMOSA has been shown to improve code coverage further than each method individually, in a new algorithm—CodaMOSA [4]. This algorithm has been adapted in Pynguin, through the name of LLMOSA.

Complete
intro

1.1 Problem

Because of the random nature of genetic algorithms, DynaMOSA has trouble generating tests for machine learning (ML) libraries, for example scikit-learn or numpy [5]. Some functions require complex data structures (*e.g.* a pandas DataFrame with a column `col_y` existing), which are hard to generate through evolution only, because that would require stumbling upon the required constraints randomly. LLMOSA seems like a promising alternative, because providing the code of the subject under test (SUT), an LLM can produce such data structures. However, when trying to run LLMOSA on code oriented toward machine learning, a few problems occurred, making the LLM unable to generate tests.

To combine the LLM and DynaMOSA approaches, Pynguin deserializes the generated code from the LLM (a sequence of characters) into a list of test case chromosomes, a format used by the DynaMOSA algorithm to evolve the test cases. This conversion (as described in figure 1) operates in two phases. The rewriter first takes the LLM response, extracts code blocks and specifically Python functions named `'test_...'`, and transforms statements to make them easier to deserialize (*e.g.* walrus expressions are transformed into regular assignments). The deserializer then converts the simplified code into Pynguin's classes.

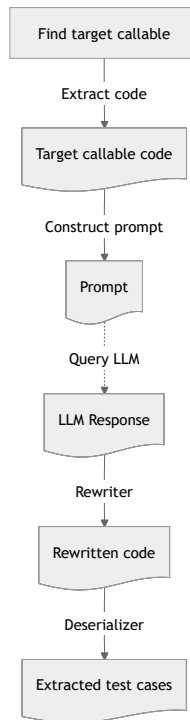


Figure 1: LLM test generation sequence

Each step adds uncertainty and potential errors. This paper aims at solving these problems one by one, first explaining them, before solving them and evaluating the impact of these changes. Our goal is to improve Pynguin’s LLMOSA algorithm, in particular—though not only—for machine learning applications.

1.2 Contributions

In this paper, we present the following contributions:

1. A system to add LLM providers—and a new provider, which allows to run open models locally, reducing the dependency on external providers and improving reproducibility.
2. A new prompt, which provides more guidance and compresses context to improve the quality of the generated tests.
3. A system to add imports from the module under test, which allows to resolve objects from the libraries used by the test suite.
4. Improvements to the deserializer, which allow to parse more statements and assertions.
5. Other various small improvements, such as handling pybind11 functions, regex improvements, and more.
6. An evaluation of the impact of these changes on the algorithm performance, in terms of code coverage and execution time.

Complete
contribu-
tions

Outline (?)

2 Background and related work

For an in depth understanding of search-based software testing (SBST), specifically for machine learning applications, we refer the reader to Chapter 2 of Lucas Berg’s thesis [6]. This paper has been written as a continuation of this thesis, and will focus on the improvements made to LLMOSA, and their evaluation, with the same goal of improving code coverage for machine learning applications.

3 Approach

Before trying to improve LLMOSA, the first step was trying to run it on a code sample. As our research is oriented toward machine learning, we wrote an example (listing 1) using pandas [7], a popular library for data analysis.

Listing 1: Module under test (MUT)

```
1 import pandas as pd
2
3 def process_df(df: pd.DataFrame) -> bool:
4     '''Applies different operations to a DataFrame, in place.'''
5     df.drop(columns=['col1'], inplace=True) # Requires col1 to exist
6     return df.empty
```

This sample has two different output to be tested, **True** and **False**. The objective is to produce both compliant and non-compliant inputs to test this module. Non compliant outputs are relatively easy to generate, as any non DataFrame input will do. However, generating a compliant input is more difficult, as it requires a DataFrame with a column named `col1`, and another non empty column. This is an example of a function that is hard to test with DynaMOSA, but *should* be easy to test with LLMOSA.

To test this, we tried running both DynaMOSA and LLMOSA on this module, with the default configuration. However, running pynguin with LLMOSA required an OpenAI API key, which has the following drawbacks:

1. Consistency: OpenAI can discontinue access to a specific model at any time, making it difficult to reproduce experiments. Open weights models can be downloaded, stored and used at any time for a better comparison and reproducibility.
2. Price: OpenAI's prices range from up to 0.40\$ per 1M output tokens for the cheapest "nano" models, and 180\$ per 1M output tokens for flagship models [8]. When benchmarking or through intensive use, this can quickly add up to substantial expenses. These prices are also subject to change.

3.1 New providers

To reduce this dependency to an external provider, we decided to refactor the LLM logic and add support for a new provider, Ollama. This provider can be used to run open models [9] locally, solving the issues raised above. A configuration entry was added; `--provider`, taking as possible values the enum `LLMProvider`—currently `OPENAI` or `OLLAMA`.

3.2 Prompt engineering

Due to the constraints of the rewriter, we noticed that the LLM often failed to provide an appropriate structure. This was primarily due to the minimalist prompt (listing 2).

Listing 2: Original prompt

```
Write unit tests for the following callables that Pynguin failed to cover:
{callables_section}
Module path: `{self.module_path}`
Module source code: `{self.module_code}`
```

To try and formulate better guidance, we listed the requirements to satisfy for the rewriter to work:

1. The answer should be inside a code block:
Pynguin uses the regex `r"```\python([\s\S]+?)(?:```|$)"` to extract the code block from the LLM response. If the answer is not inside a code block annotated with triple backticks and "python", it will not be parsed. If no code block is found, the rewriter will return an empty list of test cases, which will make the algorithm fail to generate any test case.
2. Each function must be named `'test_...'`:
The rewriter specifically checks if the function name starts with `'test_'`. If it does not, the function will be ignored and not parsed. It also has to either be top-level or inside a top-level class, but not inside another function or nested class.
3. Each function must have no arguments:
While the rewriter can handle functions with arguments, the deserializer cannot, due to the fact that Pynguin's test case representation does not support arguments. If a function has arguments, it will be ignored and not parsed. Without changing the whole test case representation, this requirement cannot be lifted.
4. Pytest features (*e.g.* fixtures) are not supported:
Again, due to the limitations of Pynguin's test case representation, the deserializer cannot handle pytest features. Decorators in general are not rewritten, and will be ignored (except for `@pytest.mark.parametrize`, as explained below).
5. ...

We added a bit of context, and complementary instructions such as:

1. Not rewriting the SUT's code, for three main reasons: first, it would skew pynguin's code coverage results, as the tests would not run on the SUT's actual code but on the rewritten version; second, it would make the tests less maintainable, as they would have to be updated if the SUT changes; third, it saves tokens and run time, which is beneficial for the overall efficiency of the testing process.

Complete
and ex-
plain why
these re-
quirements

2. Following the Arrange, Act, Assert pattern, which is a common pattern in unit testing. It helps to structure the tests and makes them easier to read and understand, which is useful for debugging and guides the LLM toward generating higher quality tests.

All these instructions were compiled into a new prompt (listing 3), with an emphasis on the fact that the functions should have no arguments, because this was the main cause of parsing failures, and an example of a valid test function.

To help with running the same test with different values, we also added support for `pytest.parametrize`. We rewrite the decorated function as one function per parameter, with the parameter initialized at the start of the function body.

Update
prompt

Listing 3: Our prompt proposition

```
You are writing tests to be used as seed for a SBST algorithm. Your goal is to
→ improve as much as possible the coverage of the tests.
Write unit tests for the following callables that Pynguin failed to cover:
{callables_section}
Module path: `{self.module_path}`
Module source code: `{self.module_code}`
You answer will be parsed for mutations, so here are the guidelines you need to
→ follow:
- Answer in a code block only using; one function for each test case, with NO
→ ARGUMENTS, NO HELPER FUNCTIONS AND NO CLASSES.
- If needed, instantiate vars in the body of the test func or use
→ pytest.parametrize, but DO NOT USE ANY OTHER PYTEST FEATURE (e.g. DO NOT
→ USE FIXTURES), as that will make the parsing fail.
- Do not rewrite the SUT's code in the tests. If you want for example to call a
→ function or instanciate a class, import it.
- Don't explain what you do, just answer in simple, concise assertion tests,
→ split in small functions, following the Arrange, Act, Assert pattern.

Here are some examples; *NEVER DO*:
def func_all_tests(param):
    var0 = param
    ...

*INSTEAD DO*:
from module_to_test import func
def test_feat1():
    var0 = func()
    assert ...

REMEMBER: NO ARGUMENTS IN YOUR TEST FUNCTIONS'
```

Naturally, the question that arises is the following:

RQ1: *Can our prompt improve the algorithm performance?*

Our prompt uses more tokens than the original one, however it is more specific and provides more guidance to the LLM. It is therefore expected to improve the algorithm performance, but it is not guaranteed. We will try to find out and answer this question in section 4.

Add
example of
rewritten
pytest.
parametrize

3.3 Regex

As mentionned above, the rewriter uses a regex (`r"```python([\s\S]+?)(?:```\$)"`) to extract the code block from the LLM response. We modified this regex to make it more robust, and to allow for more variations of the code block. The new regex is `r"```py(?:thon)?([\s\S]+?)(?:```\$)"`, which allows for the code block to be annotated with both “python” and “py”, as both are considered correct, and the LLM often uses the latter.

3.4 Adding imports

After adding the new prompt, we were able to consistently generate correct rewritten tests. When deserializing though, lots of statements got lost in the process: function calls, some asserts, attributes, some comparisons, *etc.*

To illustrate this, we show a sample of the rewritten output (listing 4) and the deserializer result (listing 5).

Listing 4: Rewritten output (sample)

```
1 def test_process_df_with_missing_values():
2     var_0 = 'Test for a DataFrame with missing values.'
3     var_1 = 'col1'
4     var_2 = 'col2'
5     var_3 = 1
6     var_4 = 2
7     var_5 = None
8     var_6 = [var_3, var_4, var_5]
9     var_7 = 3
10    var_8 = [var_3, var_4, var_7]
11    var_9 = {var_1: var_6, var_2: var_8}
12    df = pd.DataFrame(var_9)
13    var_10 = process_df(df)
14    var_11 = True
15    var_12 = var_10 is var_11
16    assert var_12
```

The deserializer result is missing many statements from the rewritten output, which is a problem for the algorithm's performance. The most important missing statements are the function calls, which are the main part of the test case. Without them, the test case is useless as it does not execute a single line of code from the SUT.

Listing 5: Deserializer result

```
1 def test_generated_function():
2     str_0 = 'Test for a DataFrame with missing values.'
3     str_1 = 'col1'
4     str_2 = 'col2'
5     int_0 = 1
6     int_1 = 2
7     none_type_0 = None
8     list_0 = [int_0, int_1, none_type_0]
9     int_2 = 3
10    list_1 = [int_0, int_1, int_2]
11    dict_0 = {str_1: list_0, str_2: list_1}
12    bool_0 = True
```

After further investigation, we figured that the main cause of parsing failures was the impossibility to resolve objects from the libraries used by the test suite, including from the MUT (*e.g.* `pd.DataFrame`, `process_df`). Each unparsed statement then in turn caused the parsing of the subsequent statements to fail. Two options were considered:

1. Gather the imports written by the LLM, import the corresponding libraries and search for matches in the imported objects.
2. Search in the libraries imported from the module under test (MUT).

Option 1 makes sure that libraries explicitly cited by the LLM are imported, but has other implications, such as making sure that the package is installed, performance issues, and more unpredictable behavior. That is why we opted for option 2. This choice is further justified by the fact that most of the time, it is not required to import libraries other than those used by the MUT. We still added the imports hinted by the LLM in the rewriter, both for debugging purposes and easing the implementation of option 1 if necessary.

3.4.1 pybind11 functions

pybind11 [10] is a library that allows to create Python bindings for C++ code. It is used in many machine learning libraries, such as PyTorch, to expose C++ functions to Python. However, pybind11 functions are not regular Python functions, and would make pynguin crash when trying to retrieve their AST, because they reference an unreachable module. This was fixed by setting the module tree to `None` when the module is not found.

3.5 Statement fixes

Even after adding the imports, even though the majority of statements were deserialized correctly, some still failed to be deserialized. We identified a few causes:

3.5.1 Asserts

Asserts were not added to the test case due to an incorrect configuration. If the assertion generator was not set to `AssertionGenerator.LLM`, the deserializer would not add asserts to the test case. However, the LLM was still generating asserts, which were then ignored by the deserializer. This was fixed by simply always adding asserts to the test case.

3.5.2 Unary not

Unary not statements (*i.e.* `a = not b`) were not deserialized because the deserializer was not handling them. We added a special case checking for the not operator and creating the appropriate `ASTAssignStatement`. This single change improves the deserializer success rate significantly because the LLM often uses the not operator in its generated code, especially when asserting that a boolean value is False (*e.g.* `assert not df.empty`).

3.5.3 Equality between variable references

A problem in the implementation of the deserializer (when testing equality between variable references) was causing it to crash on some occurrences, stopping completely the test generation. The problem was later found and a patch was added independently in the main branch of Pynguin by the maintainer [11].

3.5.4 `x is y` (?)

3.6 Evaluation

After implementing all the changes above, we ask ourselves to which extent the changes impact the algorithm performance. We will try to answer the following research question:

RQ2: *Is our new generation approach more effective?*

The evaluation will be done in section 4.

3.7 Compressing context

The fixed deserializer was able to parse the rewritten output correctly, but when testing LLMOSA on a large module, we ran into issues. The LLM was often unable to generate a proper response. Sometimes it would time out, while other times it would a response that was not parsable by the rewriter or deserializer or simply unusable. The run time also increased significantly, as the LLM had to process a lot of information. We identified that the main cause of this problem was the context size. The LLM was given too much information—the entire module code, which could be very large, and the list of callables to cover, which could also be large (often the entire module). To solve this problem, we implemented a context compression algorithm.

Using the module’s AST, we can identify the callables we want to cover, and extract only the relevant code for each callable (*e.g.* the class the callable belongs to). The other callables are then removed from the prompt (and replaced with a “`# [...]`” comment to hint that they were removed). This way, the LLM is given only the relevant information for each callable. We also limited the number of callables to cover in each prompt, to avoid giving too much information to the LLM.

We consider this compression algorithm as part of the prompt engineering, as it is a way to improve the prompt by reducing the context size. As such, we will evaluate its impact on the algorithm performance in section 4 to try and answer RQ1.

4 Evaluation

Evaluate
prompt
and other
improve-
ments

4.1 Dataset

Considering the goal of this paper is to improve LLMOSA for machine learning applications, we selected a few modules from popular machine learning libraries. We selected modules from pytorch, TabGAN, and transformers, which are popular libraries for deep learning and natural language processing. They were selected to have a wide range of complexity and popularity, and to cover different aspects of machine learning, such as neural networks, tabular data generation, and natural language processing. All these libraries are part of the Awesome Python list [12], which is a curated list of Python frameworks, libraries, tools, and resources. Here are the exact modules we selected:

- `pytorch/torch/_numpy/_ndarray.py`
- `pytorch/torch/_numpy/_getlimits.py`
- `pytorch/torch/nn/attention/bias.py`
- `pytorch/torch/nn/attention/flex.attention.py`
- `Tabular-data-generation/src/_ForestDiffusion/diffusion_with_trees.class.py`
- `Tabular-data-generation/src/tabgan/adversarial_model.py`
- `Tabular-data-generation/src/_ctgan/transformer.py`
- `transformers/src/transformers/tokenization_utils_base.py`
- `transformers/src/transformers/pipelines/text_generation.py`
- `transformers/src/transformers/data/metrics/squad_metrics.py`
- `pytorch/torch/nn/modules/rnn.py`
- `pytorch/torch/_numpy/_util.py`
- `pytorch/torch/_numpy/_funcs.py`
- `pytorch/torch/_numpy/_ufuncs.py`
- `pytorch/torch/_numpy/_normalizations.py`

4.2 Experiment setup

We want to answer the following research questions:

RQ1: *Can our prompt improve the algorithm performance?*

RQ2: *Is our new generation approach more effective?*

To do so, we ran four different experiments, two for each research question. The first experiment was run with the original prompt, and the second one with our new prompt. The third experiment was run with the original approach, and the fourth one with our new approach.

After each experiment, we collected the code coverage results and execution time, and compared them.

4.2.1 Containerization

We deployed our experiments in a compute cluster using Kubernetes. A job was created for all experiments, which created a pod for each experiment. Each pod was running a container, which contained all the dependencies required to run the experiments. The container was built using a Dockerfile, which installed all the dependencies and copied the source code of our project into the container. The Dockerfile was optimized to improve the build time.

4.2.2 Configuration

As per the recommendations of Arcuri and Briand [13] on statistical tests for assessing randomized algorithms, we ran each experiment 30 times to account for the randomness of the algorithm. The configuration of the experiments was the default configuration of Pynguin, with the following changes:

- provider: Ollama
- algorithm: LLMOSA
- model-name: See for each experiment
- call-llm-for-uncovered-callables: True
- call-llm-on-stall-detection: True
- max-plateau-len: 10
- create-coverage-report: True
-

Complete
configura-
tion details

4.3 Results

```
...
    To answer RQ2 (
        RQ2: Is our new generation approach more effective?
    ) ...
```

5 Discussion

5.1 Findings

5.2 ...

5.3 Threats to validity

6 Conclusions

6.1 Summary

6.2 Future work

Acknowledgments

I would like to thank Xavier DEVROEY (my tutor) and Lucas BERG for providing valuable guidance during my internship.

References

- [1] Gordon Fraser and Andrea Arcuri. “EvoSuite: automatic test suite generation for object-oriented software”. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ESEC/FSE ’11. Szeged, Hungary: ACM, 2011, pp. 416–419. ISBN: 978-1-4503-0443-6. DOI: 10.1145/2025113.2025179. URL: <http://doi.acm.org/10.1145/2025113.2025179>.
- [2] Stephan Lukasczyk and Gordon Fraser. “Pynguin: Automated Unit Test Generation for Python”. In: 44th International Conference on Software Engineering Companion (ICSE ’22 Companion). 2022. DOI: 10.1145/3510454.3516829.
- [3] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. “Automated Test Case Generation as a Many-Objective Optimisation Problem with Dynamic Selection of the Targets”. In: *IEEE Transactions on Software Engineering* 44.2 (2018), pp. 122–158. DOI: 10.1109/TSE.2017.2663435.
- [4] Caroline Lemieux et al. “CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2023, pp. 919–931. DOI: 10.1109/ICSE48619.2023.00085.
- [5] Nicolas Erni et al. *SBFT Tool Competition 2024 – Python Test Case Generation Track*. 2024. arXiv: 2401.15189 [cs.SE]. URL: <https://arxiv.org/abs/2401.15189>.
- [6] Lucas Berg. “Improving automated unit test generation for machine learning libraries using structured input data”. Master’s thesis. University of Namur, June 17, 2024.
- [7] The pandas development team. *pandas-dev/pandas: Pandas*. DOI: 10.5281/zenodo.3509134. URL: <https://github.com/pandas-dev/pandas>.
- [8] OpenAI Developers. *Compare models*. July 2, 2026. URL: <https://developers.openai.com/api/docs/models/compare>.
- [9] Ollama. *Ollama*. 2026. URL: <https://github.com/ollama/ollama>.
- [10] pybind11 contributors. *pybind11*. 2026. URL: <https://github.com/pybind/pybind11>.
- [11] Lukas Kroding. *Commit 55c6806*. June 30, 2026. URL: <https://github.com/se2p/pynguin/commit/55c680653adc216ef4130eef6411814b7d67f4e5>.
- [12] vinta. *Awesome Python*. July 3, 2026. URL: <https://github.com/vinta/awesome-python>.
- [13] Andrea Arcuri and Lionel Briand. “A practical guide for using statistical tests to assess randomized algorithms in software engineering”. In: *Proceedings of the 33rd International Conference on Software Engineering*. ICSE ’11. Waikiki, Honolulu, HI, USA: Association for Computing Machinery, 2011, pp. 1–10. ISBN: 9781450304450. DOI: 10.1145/1985793.1985795. URL: <https://doi.org/10.1145/1985793.1985795>.